



MATH-454: PARALLEL AND HIGH-PERFORMANCE COMPUTING

Final Project

Author:

Mattia BARBIERE (387974)

Professor:

Pablo ANTOLIN SANCHEZ

Spring Semester - 2025

1 Theoretical analysis

As a first step I present the theoretical computational analysis of the problem at hand. I take N_x and N_y to be the size of the whole grid in the x and y direction respectively. The computational cost of initializing the problem is $O(N_x N_y)$. For the water drop and dummy tsunami this comes from iterating over the for loops, while for the real tsunami this is the cost of reading from the file. Next the code performs N_T time steps, where at each time step it computes the next time step $\mathbf{dt}$, updates the boundary conditions, and performs the next step in the evolution of the PDE.¹ Computing the next time step and performing the next step both take $O(N_x N_y)$ time while updating the boundary conditions takes $O(2(N_x + N_y))$ computations. Putting everything together gives a computational cost of

$$O(N_x N_y + 2N_T (N_x N_y + N_x + N_y)). \quad (1)$$

Notice, however, that although the iterations in time cannot be parallelized, all other operations can. This is the first sign that the code is highly parallelizable.

After performing a `perf` analysis it was immediately clear that the `solve_step` and `compute_time_step` functions were the biggest bottlenecks. In fact in Equation (1) we have that the dominating term is $O(2N_T N_x N_y)$ which arises from `solve_step` and `compute_time_step` functions. Fortunately, both of these functions are good candidates for parallelization. Nonetheless, it was also very easy to parallelize the code for initializing the problem (for water drops and dummy tsunami) and the code for updating the boundary conditions.

2 MPI parallelization

When parallelizing using MPI the first step is to figure out how to subdivide the domain, which in our case is a 2D grid, among the processors. The method I chose to use was to assign every processor a 2D subgrid of points. Then, in parallel, each processor could focus on the computations within its subgrid. The difficult part of this approach is the communication between processors. This is essential since the computation for every unit in the grid depends on its neighbours. The trick for this to work is to have a halo around all four edges of each subgrid. These halos will be copies of the boundaries of the neighbouring subgrids for all four sides. The communication between processors thus reduces to coping these halos among neighbouring subgrids. Fortunately, MPI has a communicator which is used for this precise purpose called the **Cartesian communicator**. This communicator splits up the grid of processors for us, automatically assigning to each processor its neighbours simply by calling `MPI_Cart_shift`. Communicating between processors simply boils down to using `MPI_Sendrecv` and using the neighbours provided by the Cartesian communicator. The second step in the parallelization is computing the maximum over the whole grid when running the `compute_time_step` function. This can easily be done by invoking the `MPI_Allreduce` function with the `MPI_MAX` operation.

Next I present my results for strong and weak scaling using the parallelized code explained above. Note that for all the parallelization results the **water drops** test case was used with the `-O3` optimization flag. I also computed the times with `n_outputs=0` so that the time of writing to file is not considered in the scaling results.

2.1 Strong Scaling

Firstly, let us look at the theoretical analysis of my parallelization strategy when performing strong scaling. As previously mentioned, essentially every operation hindering the performance of the code is parallelizable. To subdivide the domain I divide both N_x and N_y by $\sqrt{p}$. In practice this is done with `MPI_Dims_create` which returns integer values. Hence every processor is responsible for computing the values for a subgrid of size $\frac{N_x}{\sqrt{p}} \times \frac{N_y}{\sqrt{p}}$. Running all the operations in parallel Equation (1) becomes

$$O\left(\frac{N_x N_y}{p} + 2N_T \left(\frac{N_x N_y}{p} + \frac{N_x}{\sqrt{p}} + \frac{N_y}{\sqrt{p}}\right)\right)$$

¹I assume that swapping data takes constant time as it is a matter of updating pointers.

Nonetheless, this is not the full story. Firstly, when subdividing the domain in 2D subgrids there is inevitably some communication cost that is added. Secondly, using `MPI.Allreduce` when computing the time step also affects cost of parallelizing.

In fact every processor has to communicate $O\left(\frac{2N_x}{\sqrt{p}} + \frac{2N_y}{\sqrt{p}}\right)$ values in total with its neighbours. Thus the extra cost of communicating among all processors is ²

$$O\left(p\left(\frac{2N_x}{\sqrt{p}} + \frac{2N_y}{\sqrt{p}}\right)\right) = O(2\sqrt{p}(N_x + N_y)).$$

Lastly, the communication cost of performing `MPI.Allreduce` is $O(\log p)$. Putting everything together we get an overall complexity of

$$O\left(\underbrace{\frac{N_x N_y}{p} + 2N_T\left(\frac{N_x N_y}{p} + \frac{N_x}{\sqrt{p}} + \frac{N_y}{\sqrt{p}}\right)}_{\text{Computation complexity}} + \underbrace{2\sqrt{p}(N_x + N_y)}_{\text{Neighbour communication}} + \underbrace{\log p}_{\text{MPI.Allreduce}}\right). \quad (2)$$

Given this complexity bound, we expect that for low processor counts the dominating term remains the computational part of the code as each processor will still have a lot of computations to perform and relatively little communication. However, when p becomes large then the communication part will start dominating. Indeed when we have large values of p each processor will have a smaller subgrid on which to perform calculations, however we will also have a lot more processors each of which will have to communicate with its neighbours making the communication part more expensive.

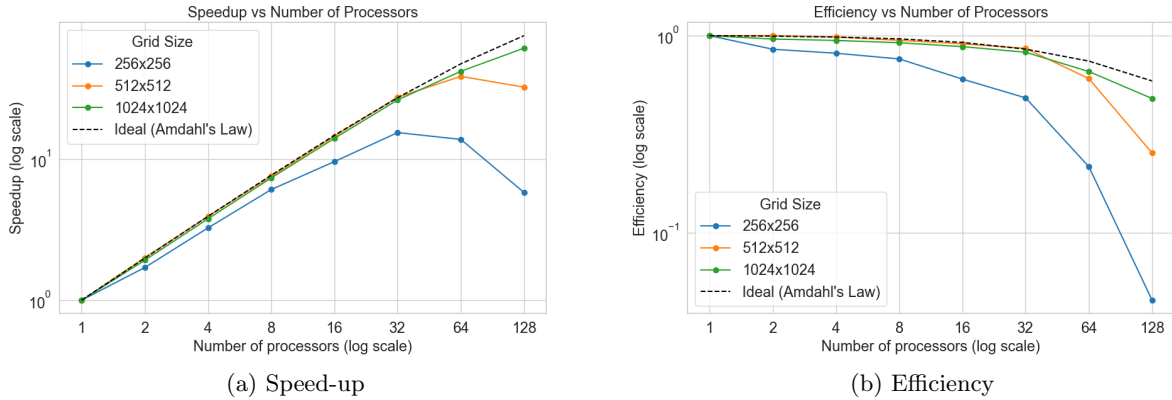


Figure 1: Speed-up and efficiency when performing strong scaling. Strong scaling was performed on three sizes of grids. The black dashed line represents the ideal speed-up and efficiency given by Amdahl's law with $\alpha = 99.45\%$.

To see how well the code performs under strong scaling we need to check what is the ideal scaling according to Amdahl's law. After running a performance analysis with `perf` I estimated alpha to be approximately $\alpha = 99.45\%$. This agrees with our observation that the code is highly parallelizable. Amdahl's law thus tells us the the ideal speed-up is $S(p) = \frac{1}{(1-99.45\%) + \frac{99.45\%}{p}}$. Plotting this value against the code performance we get the graph in Figure 1a. Similarly, the efficiency $E(p) = S(p)/p$ is shown in Figure 1b. These figures align very well with the theoretical analysis. Indeed when the number of processors is low all the curves are very close to the ideal value of speed-up and efficiency. Conversely, when the number of processors increases the communication cost becomes the dominant term and all the curves deviate from Amdahl's law. Note that larger grid sizes deviate later on because, for a given value of p , the subgrids remain larger and thus the computation part dominants for longer.

²The subgrids that are on the boundary have fewer neighbours but I provide an upper bound for the communication complexity.

2.2 Weak Scaling

To perform weak scaling I need to make sure that each processor has the same amount of work to perform. With this goal in mind, I fixed n_x and n_y and then computed the global grid size as $N_x = n_x\sqrt{p}$ and $N_y = n_y\sqrt{p}$. In doing so I notice that each processor gets a subgrid of $\frac{N_x}{\sqrt{p}} \times \frac{N_y}{\sqrt{p}} = n_x \times n_y$ and clearly this value is independent of the number of processors. Recalling Equation (2), we can substitute in the new values for N_x and N_y and get

$$O \left(\underbrace{n_x n_y + 2N_T (n_x n_y + n_x + n_y)}_{\text{Computation complexity}} + \underbrace{2p(n_x + n_y)}_{\text{Neighbour communication}} + \underbrace{\log p}_{\text{MPI_Allreduce}} \right). \quad (3)$$

The striking thing here is that, when performing weak scaling, the communication with the neighbours now scales with p while the computational cost is essentially constant.

Having done this theoretical analysis, I used Gustafson's law to understand the ideal weak scaling. The law states that the ideal speed-up should be $S(N, p) = 1 + (p - 1)\alpha(N)$. As a simplifying assumption, I assumed that $\alpha(N)$ is constant. After performing an analysis using `perf` I estimated $\alpha(N) = \alpha = 96.51\%$. With this threshold in mind, I performed weak scaling on my code with three values of local grid sizes as shown in Figure 2. To make the experiments easier, I only chose values of p which are perfect squares so that N_x and N_y could be computed easily.

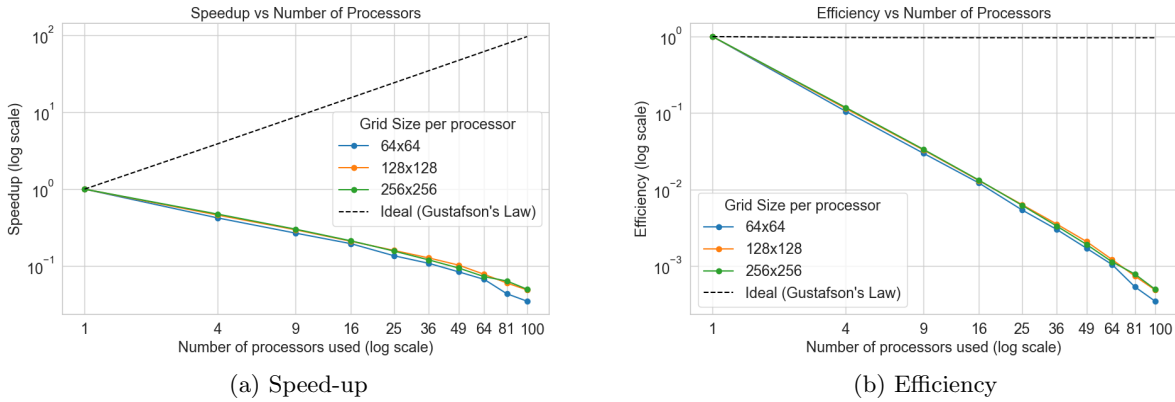


Figure 2: Speed-up and efficiency when performing weak scaling. Weak scaling was performed on three sizes of grids per processor with $n_x = n_y = 64$ (blue), $n_x = n_y = 128$ (orange), and $n_x = n_y = 256$ (green). The black dashed line represents the ideal speed-up and efficiency given by Gustafson's law with $\alpha(N) = \alpha = 96.51\%$.

After inspection of Figure 2 I notice that the plots are very much in line with expectations. Indeed, since the plot is using log-log axes the straightness of the lines agrees with Equation (3) where we expected the computational cost to scale with p . We also notice that this scaling is more or less the same throughout the plots, and indeed Equation (3) tells us that the computational part of the code should be essentially constant and thus should not affect scaling.

3 CUDA

Parallelization in CUDA is conceptually very similar to the parallelization done with MPI. The idea is again to subdivide the grid into blocks and allow each block to perform the calculations in parallel within its assigned region. All the data for the calculations is stored in the device's global memory thus, there is no need for communication between blocks. Similarly to the previous section, the best candidates for parallelization were the `solve_step` and `compute_time_step` functions. While the `solve_step` function was very easy to parallelize since it is simply two nested for loops the `compute_time_step` was trickier. The idea is that this

function must compute the a maximum value over the entire grid, however CUDA does not have something equivalent to `MPI_Allreduce`. To solve this issue I used a NVIDIA presentation available [here](#) as inspiration. To perform the reduction operation I divided the computation into two CUDA kernels. The first kernel would compute the maximum for every block and write the values to global memory. It computed this by allowing each thread of a block to write its value in shared memory and once all the threads had finished, it iterated over the shared memory data until it had found the maximum over the single block. Then the second kernel would compute the maximum of all block maxima stored in global memory resulting in the maximum value over the global grid.

After parallelizing these functions the next bottleneck was the `update_bcs` function which was also easy to parallelize. I also chose to parallelize the `init_gaussian` and `init_dummy_tsunami` since they were very simple to run in parallel.

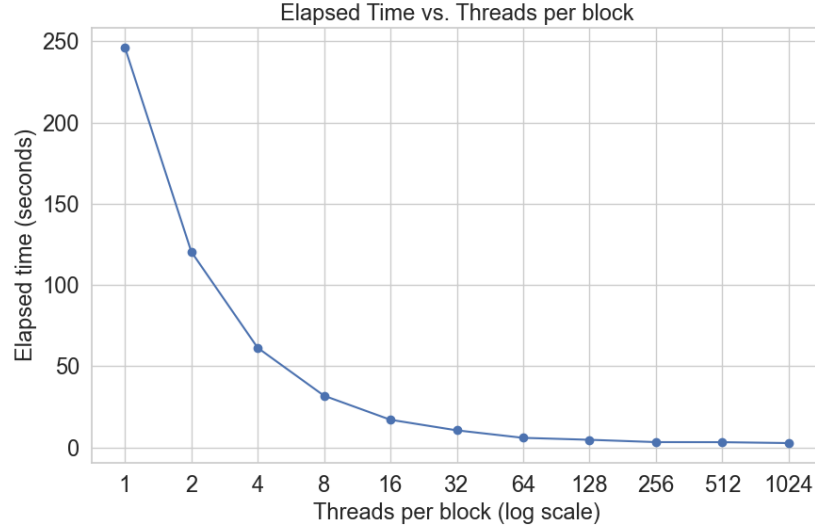


Figure 3: Time of executing code versus number of threads per block when parallelizing using CUDA. The code was run on a grid size of 1000×1000 when solving the **water drops** problem.

Figure 3 illustrates the time for solving the shallow water equations with parallelized code as explained above. I observe that initially the increase in threads per block dramatically helps the computational time. However, as the number of threads per block is increased further the marginal decrease in time becomes smaller until it eventually reaches a plateau. This is certainly explained by the fact that, when working with a large amount of threads, the computing part of the code will be performed very fast, and eventually the bottlenecks becomes the non-parallelizable part of the code which, while being a small amount, might become important when using a lot of threads per block. Other improvements might also help the code run faster. For example the NVIDIA presentation [here](#) suggest more advanced and quicker ways of performing the maximum reduction. I believe however, that the main bottleneck is indeed the serial part of the code which imposes a hard limit on the achievable speed.

4 Conclusion

In this report I demonstrated that the shallow water equations have a lot of parallelization potential. The theoretical analysis of the code showed that many of the slowest functions were perfect candidates for parallelization with both with MPI and CUDA. This is also reflected in the results presented. Both strong scaling with MPI and the scaling in CUDA showed very good results, with the inevitable diminishing returns caused by communication overhead and serial parts of the code. Weak scaling on the other hand, performs less well since the communication cost immediately dominates the computational complexity which is, nevertheless, very much inline with the theoretical analysis. Overall, both in theory and in practice the shallow water equations gain a lot when run in parallel.